

SHETH L.U.J COLLEGE

AI PRACTICAL1

A) Implement the Breadth First Search algorithm to solve a given problem.

CODE

```
AI_Prac1.1.py - D:/daniyal/AI_Prac1.1.py (3.14.6)
File Edit Format Run Options Window Help

from collections import deque

graph = {
    "Mumbai": ["Dubai", "Doha"],
    "Dubai": ["Cairo", "Istanbul"],
    "Doha": ["Paris"],
    "Cairo": ["London"],
    "Istanbul": ["London"],
    "Paris": ["London"],
    "London": []
}

def bfs(graph, start, goal):
    visited = set()
    queue = deque([[start]])

    while queue:
        path = queue.popleft()
        city = path[-1]

        if city == goal:
            return path

        if city not in visited:
            visited.add(city)

            for neighbour in graph[city]:
                new_path = list(path)
                new_path.append(neighbour)
                queue.append(new_path)

    return None

start = "Mumbai"
goal = "London"

result = bfs(graph, start, goal)

if result:
    print("Route found using BFS:")
    print(" -> ".join(result))
else:
    print("No route found.")

Ln: 45 Col: 0
```

OUTPUT

```
IDLE Shell 3.14.6
File Edit Shell Debug Options Window Help

Python 3.14.6 (tags/v3.14.6:1c63e6c, Jun 10 2026, 10:26:10) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>> ===== RESTART: D:/daniyal/AI_Prac1.1.py =====
Route found using BFS:
Mumbai -> Dubai -> Cairo -> London
>>>
```

SHETH L.U.J COLLEGE

AI PRACTICAL1

B) Implement the Iterative Depth First Search algorithm to solve the same problem.

CODE

```
*AI_Prac1.2.py - D:/daniyal/AI_Prac1.2.py (3.14.6)
File Edit Format Run Options Window Help

graph = {
    "Mumbai": ["Dubai", "Doha"],
    "Dubai": ["Cairo", "Istanbul"],
    "Doha": ["Paris"],
    "Cairo": ["London"],
    "Istanbul": ["London"],
    "Paris": ["London"],
    "London": []
}

def dfs(graph, start, goal):
    visited = set()
    stack = [start]

    while stack:
        path = stack.pop()
        city = path[-1]

        if city == goal:
            return path

        if city not in visited:
            visited.add(city)

            for neighbour in reversed(graph[city]):
                new_path = list(path)
                new_path.append(neighbour)
                stack.append(new_path)

    return None

start = "Mumbai"
goal = "London"

result = dfs(graph, start, goal)

if result:
    print("Route found using Iterative DFS:")
    print(" -> ".join(result))
else:
    print("No route found.")
```

Ln: 42 Col: 28
27°C Light rain 10:21 PM 6/30/2026

OUTPUT

```
IDLE Shell 3.14.6
File Edit Shell Debug Options Window Help

Python 3.14.6 (tags/v3.14.6:c63aee6, Jun 10 2026, 10:26:10) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>> ===== RESTART: D:/daniyal/AI_Prac1.2.py =====
Route found using Iterative DFS:
Mumbai -> Dubai -> Cairo -> London
>>>
```

Ln: 7 Col: 0
27°C Light rain 10:21 PM 6/30/2026

SHETH L.U.J COLLEGE

AI PRACTICAL1

C) Compare the performance and efficiency of both algorithms.

The BFS and Iterative DFS algorithms were successfully implemented to find a route from Mumbai to London. BFS finds the shortest path by exploring nodes level by level, while DFS explores one path deeply before backtracking. Both algorithms have a time complexity of $O(V + E)$, and the choice between them depends on the problem requirements.